
Bölüm-6

NESNEYE DAYALI PROGRAMLAMA “ÇOKBİÇİMLİLİK VE ARAYÜZLER”



Bu bölümde;

- ▶ **Bu bölümde çok-biçimlilik (polymorphism) konseptini öğreneceksiniz.**
- ▶ **Çok biçimlilik için üzerine yazılmış (overridden) metotlar kullanacaksınız.**
- ▶ **“abstract” ve “concrete” sınıflar arasındaki farkı öğreneceksiniz.**
- ▶ **“Abstract” sınıflar ve metotlar bildireceksiniz.**
- ▶ **Çok-biçimliliğin sistemleri nasıl genişletilebilir ve yönetilebilir yaptığını öğreneceksiniz.**
- ▶ **Çalışma zamanı nesnelere karar vereceksiniz.**
- ▶ **Arayüz (Interface) bildirecek ve kodlayacaksınız.**



- ▶ Giriş
- ▶ Çok-biçimlilik Örnekleri
- ▶ Çok-biçimlilik Davranışlarının Gösterilmesi
- ▶ “Abstract” sınıflar ve metotlar
- ▶ Case Study: “Çok-biçimlilik ile Bordro Sistemi”
- ▶ “final” metotlar ve sınıflar
- ▶ Constructor içinde metot çağırma probleminin açıklanması
- ▶ Arayüz (interface) tanımlama
- ▶ JAVA 8 Interface
- ▶ JAVA 9 “private interface” ve “private constructor”



- Çok-biçimlilik;
- “Özel-program” yerine “genel-program” yazmaya imkan tanır.
- Aynı üst sınıfı paylaşan nesneleri;
 - Hepsi üst sınıfın nesneleriymiş gibi işleyen programlar yapmanızı sağlar.
- (Bu da kodlamayı kolaylaştırır.)





► Örnek: Biyolojik bir çalışma için çeşitli hayvan türlerinin hareketini simüle eden bir program oluşturduğumuzu varsayalım.

► Balık, Kurbağa ve Kuş sınıfları, araştırılan üç hayvan türünü temsil eder.

► Her sınıf

ANIMALS



► Bir “move” metodu içerir. (Alt sınıflar “move” metodunu kendisi kodlar.)

► Bir hayvanın mevcut konumunu x-y koordinatları olarak saklayan “Animal” üst sınıfını miras alır.

► Bir program

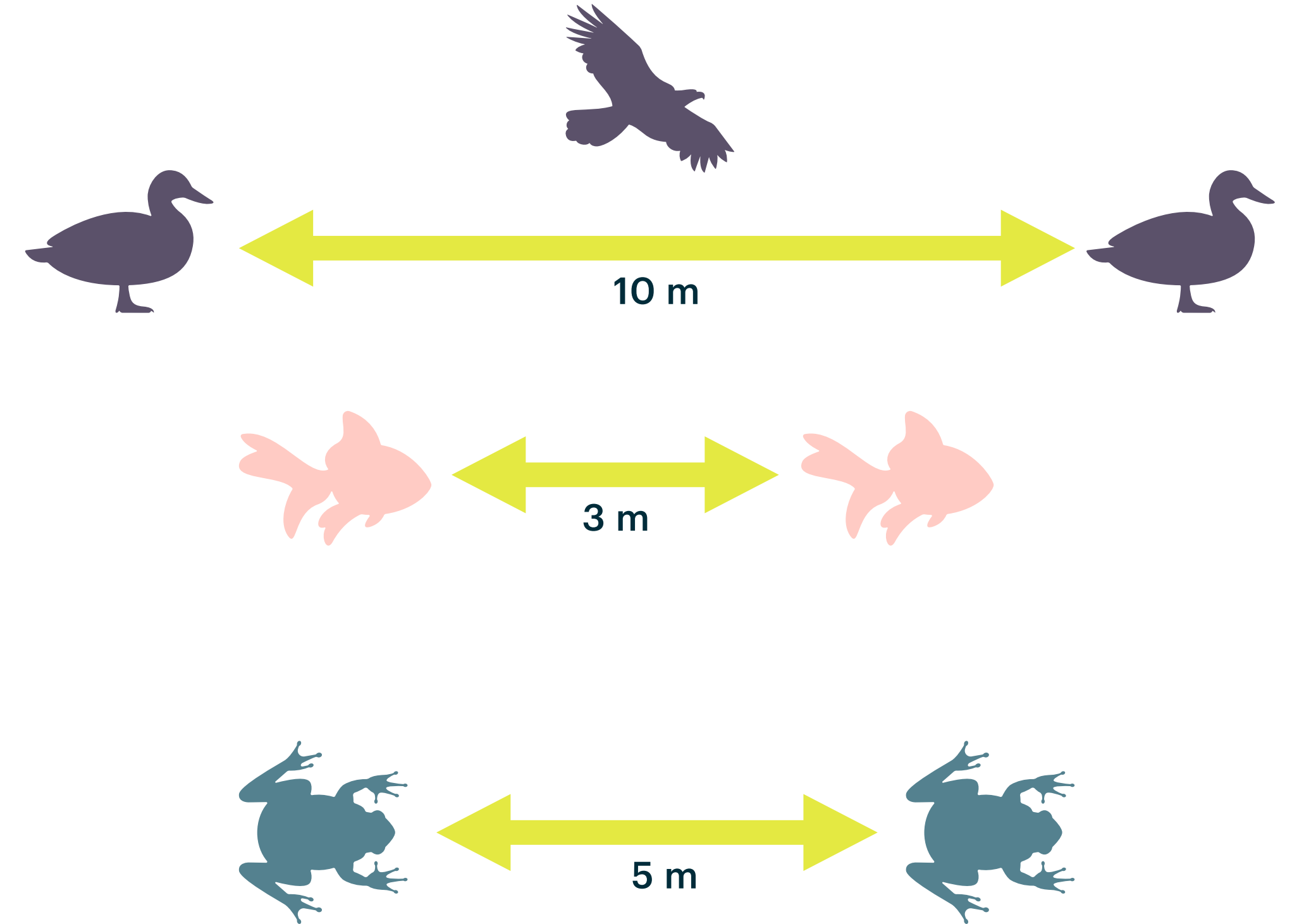
► Çeşitli Animal alt sınıflarının nesnelere referanslar içeren bir Animal dizisi tutar.

► Hayvanların hareketlerini simüle etmek için program her bir nesneye saniyede bir kez aynı mesajı gönderir. “MOVE!”



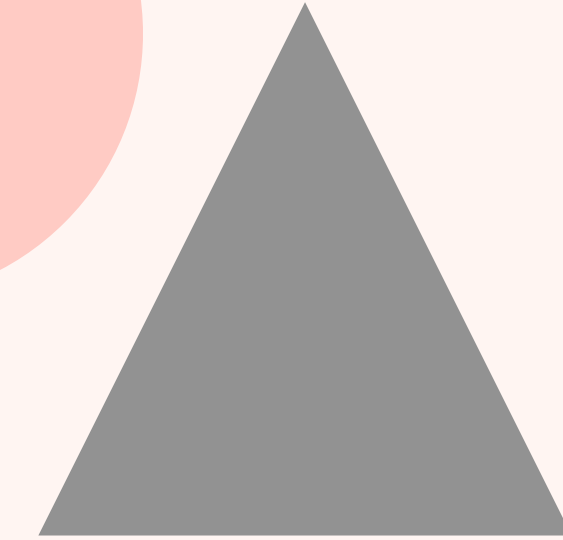
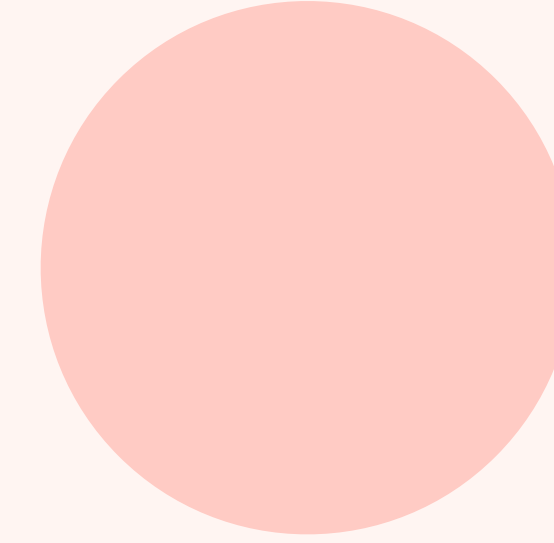
- Her Hayvan türü, bir "MOVE" mesajına benzersiz bir şekilde yanıt verir:
- Bir Balık üç metre yüzebilir.
- Bir Kurbağa beş metre zıplayabilir.
- Bir Kuş on metre uçabilir.
- Program her bir hayvan nesnesine aynı mesajı (yani MOVE) verir, ancak her nesne kendi özel hareket tipine uygun olarak x-y koordinatlarını nasıl değiştireceğini bilir.
- Aynı metod çağrısına yanıt olarak "doğru olanı yapmayı" bilmek için her bir nesneye güvenmek, çok-biçimlilik anahtar kavramıdır.

MOVE:



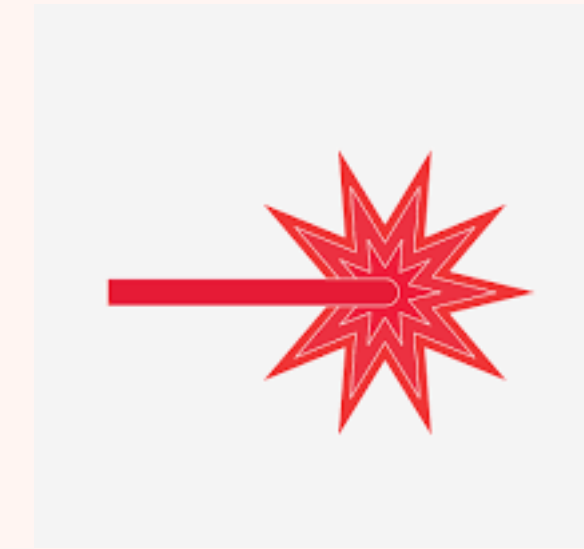
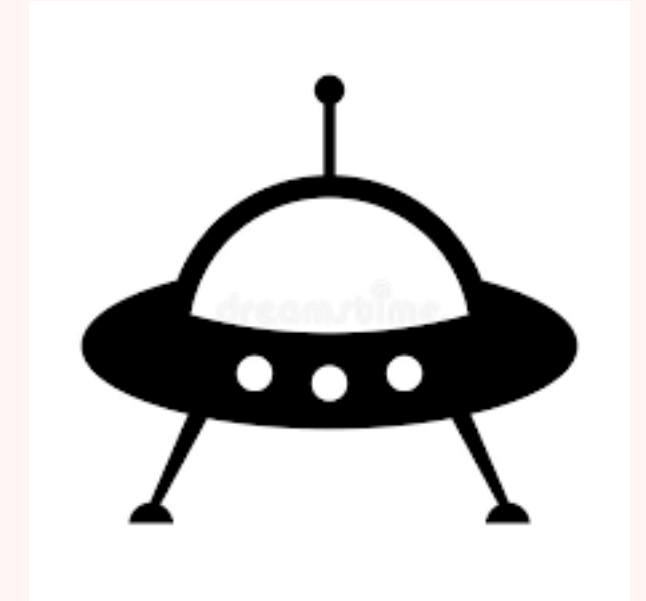
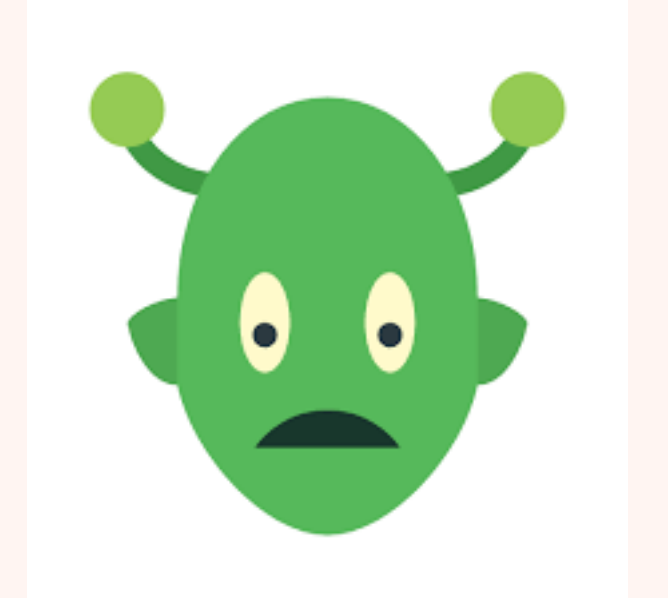


- ▶ Dikdörtgen, Dörtgen'den türetilmişse, bir Dikdörtgen nesnesi, Dörtgen'in daha spesifik bir versiyonudur.
- ▶ Dörtgen üzerinde yapılabilecek herhangi bir işlem Dikdörtgen üzerinde de yapılabilir.
- ▶ Bu işlemler Kareler, Paralelkenarlar ve Yamuklar gibi diğer Dörtgenler üzerinde de yapılabilir.
- ▶ Çok biçimlilik, bir program bir üst sınıf Dörtgen değişkeni aracılığıyla bir yöntemi çağırdığında ortaya çıkar;
- ▶ Yürütme zamanında, üst sınıf değişkeninde depolanan referansın türüne göre yöntemin doğru alt sınıf sürümü çağrılır.



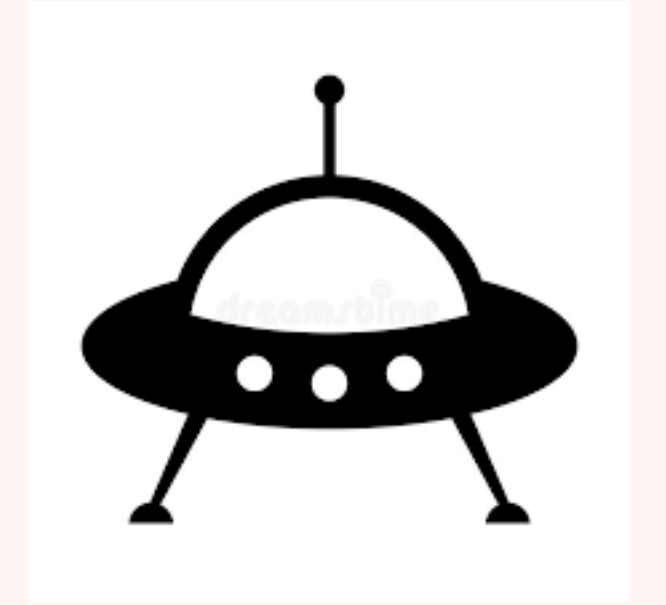
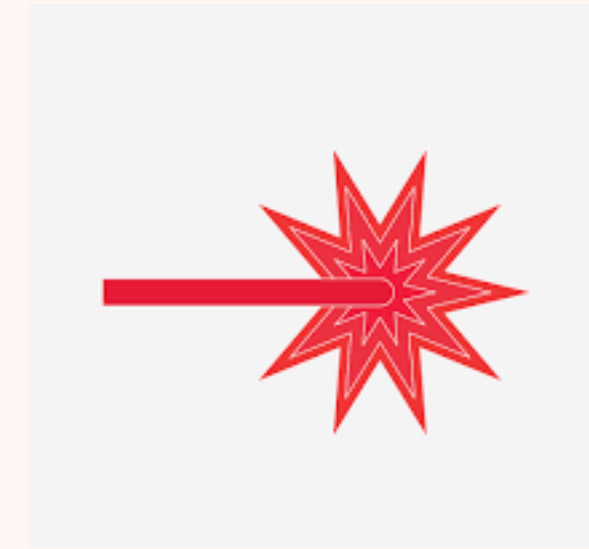
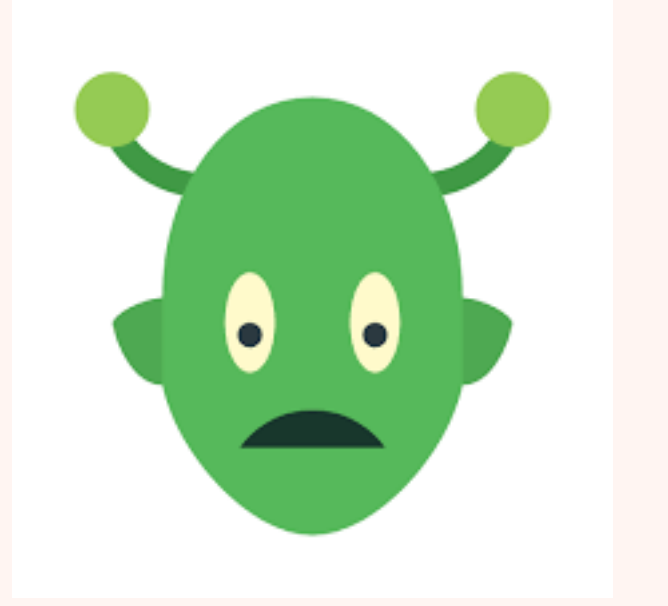


- ▶ Örnek: Bir Video Oyunundaki Uzay Nesneleri
- ▶ Bir video oyunu;
 - ▶ Martian, Venusian, Plutonian, SpaceShip ve LaserBeam sınıflarının nesnelerini manipüle eder.
 - ▶ Her biri SpaceObject öğesinden miras alır ve kendi DRAW metodunu OVERRIDE eder.
- ▶ Bir ekran yöneticisi, çeşitli sınıfların nesnelerine ilişkin bir referans dizisini tutar,
 - ▶ Her nesneye periyodik olarak aynı mesajı, yani “DRAW” metodunu gönderir.
 - ▶ Her nesne benzersiz bir şekilde yanıt verir.





- Bir Mars nesnesi; siyah gözlerle ve uygun sayıda antenle kendisini yeşile boyayabilir.
- Bir SpaceShip nesnesi; kendisini parlak siyah bir uçan daire olarak çizebilir.
- Bir LaserBeam nesnesi; kendisini ekran boyunca parlak kırmızı bir ışın olarak çizebilir.
- Çeşitli nesnelere gönderilen aynı mesajın (bu durumda, DRAW) “birçok biçimi” vardır.
- Her alt sınıf “DRAW” metodunu kendisi tekrar kodlar.



SOFTWARE ENGINEERING OBSERVATION



Çok-biçimlilik, genellemelerle uğraşmanızı ve yürütme zamanı ortamının ayrıntıları çözmenizi sağlar.

Aynı kalıtım hiyerarşisine ait oldukları sürece, nesnelere onların türlerini bilmeden kendilerine uygun davranışlar sergilemelerini söyleyebilirsiniz.

SOFTWARE ENGINEERING OBSERVATION



Çok biçimlilik genişletilebilirliği destekler: Çok biçimli davranışı başlatan yazılım, iletilerin gönderildiği nesne türlerinden bağımsızdır.

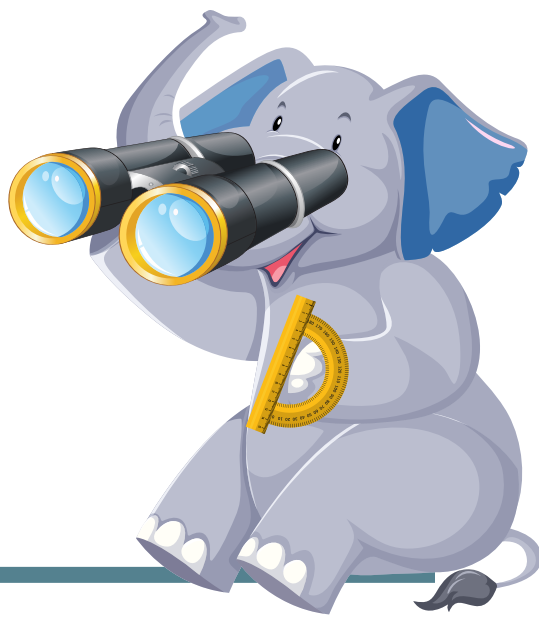
Mevcut yöntem çağrılarına yanıt verebilen yeni nesne türleri, temel sistemi değiştirmeden bir sisteme dahil edilebilir.

Yalnızca yeni nesneleri somutlaştıran istemci kodu, yeni türleri barındıracak şekilde değiştirilmelidir.



- ▶ Bu örnekte, bir alt sınıf nesnesine bir üst sınıf referansı vereceğiz.
- ▶ Bir üst sınıf referansı aracılığıyla bir alt sınıf nesnesi üzerinde bir metodu çağırmak, alt sınıf işlevselliğini çağırır.
- ▶ **Değişkenin türü değil, başvuru nesnenin türü, hangi yöntemin çağrılacağını belirler.**
- ▶ Bu örnek, bir alt sınıfın bir nesnesinin, çeşitli ilginç manipülasyonlara izin vererek, üst sınıfının bir nesnesi olarak ele alınabileceğini göstermektedir.
- ▶ Bir üst sınıf nesnesi, bir alt sınıf nesnesi olarak ele alınamaz, çünkü bir üst sınıf nesnesi, alt sınıflarından herhangi birinin nesnesi değildir.
- ▶ “is-a” ilişkisi yalnızca bir alt sınıftan onun doğrudan (ve dolaylı) üst sınıflarına kadar olan hiyerarşide geçerlidir.
- ▶ Java derleyicisi, üst sınıf başvurusunu açıkça alt sınıf türüne aktarırsanız, bir alt sınıf değişkenine bir üst sınıf başvurusunun atanmasına izin verir;
- ▶ Bir programın üst sınıfta olmayan alt sınıf yöntemlerini çağırmasını sağlayan, aşağı yayın (downcasting) olarak bilinen bir tekniktir.

SOFTWARE ENGINEERING OBSERVATION



**Her ne kadar izin verilse de
down-casting kullanmaktan kaçının.**



```
1 // Fig. 10.1: PolymorphismTest.java
2 // Assigning superclass and subclass references to superclass and
3 // subclass variables.
4
5 public class PolymorphismTest {
6     public static void main(String[] args) {
7         // assign superclass reference to superclass variable
8         CommissionEmployee commissionEmployee = new CommissionEmployee(
9             "Sue", "Jones", "222-22-2222", 10000, .06);
10
11         // assign subclass reference to subclass variable
12         BasePlusCommissionEmployee basePlusCommissionEmployee =
13             new BasePlusCommissionEmployee(
14                 "Bob", "Lewis", "333-33-3333", 5000, .04, 300);
```

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part I of 5.)



```
15
16 // invoke toString on superclass object using superclass variable
17 System.out.printf("%s %s:%n%n%s%n",
18     "Call CommissionEmployee's toString with superclass reference ",
19     "to superclass object", commissionEmployee.toString());
20
21 // invoke toString on subclass object using subclass variable
22 System.out.printf("%s %s:%n%n%s%n",
23     "Call BasePlusCommissionEmployee's toString with subclass",
24     "reference to subclass object",
25     basePlusCommissionEmployee.toString());
26
27 // invoke toString on subclass object using superclass variable
28 CommissionEmployee commissionEmployee2 =
29     basePlusCommissionEmployee;
30 System.out.printf("%s %s:%n%n%s%n",
31     "Call BasePlusCommissionEmployee's toString with superclass",
32     "reference to subclass object", commissionEmployee2.toString());
33 }
34 }
```

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part 3 of 5.)



```
15
16 // invoke toString on superclass object using superclass variable
17 System.out.printf("%s %s:%n%n%s%n%n",
18     "Call CommissionEmployee's toString with superclass reference ",
19     "to superclass object", commissionEmployee.toString());
20
21 // invoke toString on subclass object using subclass variable
22 System.out.printf("%s %s:%n%n%s%n%n",
23     "Call BasePlusCommissionEmployee's toString with subclass",
24     "reference to subclass object",
25     basePlusCommissionEmployee.toString());
26
```

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part 2 of 5.)

```
27 // invoke toString on subclass object using superclass variable
28 CommissionEmployee commissionEmployee2 =
29     basePlusCommissionEmployee;
30 System.out.printf("%s %s:%n%n%s%n",
31     "Call BasePlusCommissionEmployee's toString with superclass",
32     "reference to subclass object", commissionEmployee2.toString());
33 }
34 }
```

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part 3 of 5.)

Call CommissionEmployee's toString with superclass reference to superclass object:

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 10000.00
commission rate: 0.06

Call BasePlusCommissionEmployee's toString with subclass reference to subclass object:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part 4 of 5.)

Call BasePlusCommissionEmployee's toString with superclass reference to subclass object:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00



- Bir üst sınıf değişkeni, bir alt sınıf nesnesine bir başvuru içerdiğinde ve bu başvuru bir metodu çağırarak için kullanıldığında, metodun alt sınıf sürümü çağrılır.
- Java derleyicisi bu "çaprazlamaya" izin verir, çünkü bir alt sınıfın nesnesi, üst sınıfının da bir nesnesidir (ancak tersi değil).
- Derleyici, bir değişken aracılığıyla yapılan bir metod çağrısıyla karşılaştığında, değişkenin sınıf türünü kontrol ederek metodun çağrılıp çağrılmayacağını belirler.
- Bu sınıf uygun metod bildirimini içeriyorsa (veya birini devralırsa), çağrı derlenir.
- Yürütme zamanında, değişkenin başvurduğu nesnenin türü, kullanılacak gerçek metodu belirler.
- Bu işleme dinamik bağlama (dynamic binding) denir.

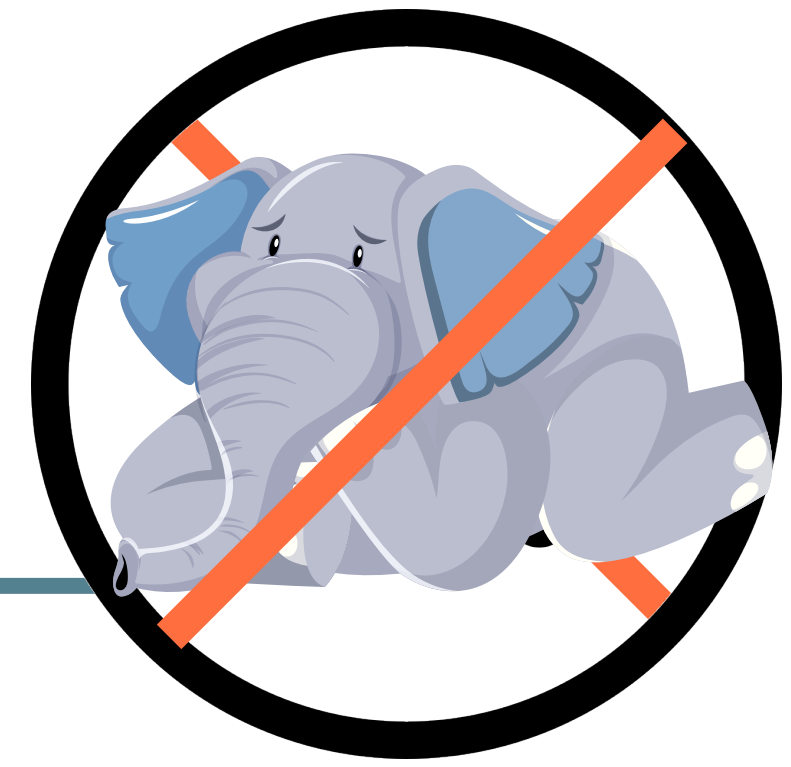


- ▶ Soyut sınıflar (abstract classes);
 - ▶ Bazen hiç bir zaman nesnesi olmayacak sınıfları bildirmek yararlı olabilir.
 - ▶ Kalıtım hiyerarşilerinde yalnızca üst sınıflar olarak taslak olurlar, bu nedenle "soyut üst sınıflar" olarak adlandırılırlar.
 - ▶ Nesneleri başlatmak için kullanılamaz -> soyut sınıflar eksiktir.
 - ▶ Alt sınıflar, nesneleri somutlaştırabileceğiniz "somut" sınıflar olmak için "eksik parçaları" bildirmelidir; aksi takdirde bu alt sınıflar da soyut olacaktır.
- ▶ Soyut bir sınıf, diğer sınıfların miras alabileceği ve böylece ortak bir tasarımı paylaşabileceği bir üst sınıf sağlar.
- ▶ Somut sınıflar (concrete classes);
 - ▶ Nesneleri başlatmak için kullanılabilen sınıflara denir.
 - ▶ Bu tür sınıflar, bildirdikleri her yöntemin uygulamalarını sağlar (bazı uygulamalar miras alınabilir).
- ▶ Tüm hiyerarşiler soyut sınıflar içermez.



- ▶ “abstract” anahtar kelime ile bildirerek bir sınıf soyut yaparsınız.
- ▶ Soyut bir sınıf normalde bir veya daha fazla soyut yöntem içerir.
- ▶ Soyut bir yöntem, aşağıdaki gibi, bildiriminde “abstract” anahtar sözcüğü olan bir örnek yöntemidir.
 - ▶ `public abstract void draw(); //soyut metot`
- ▶ Soyut metotların uygulaması olmaz (içerisinde kod yok).
- ▶ Eğer bir sınıf somut metotların yanında soyut metotları da içeriyorsa o sınıf soyut bir sınıf olmalıdır.
- ▶ Soyut bir üst sınıfın her bir somut alt sınıfı, aynı zamanda, üst sınıfın soyut yöntemlerinin her birinin somut uygulamalarını sağlamalıdır. (Derleyici sizi uyaracaktır.)
- ▶ Yapıcılar ve statik yöntemler soyut olarak bildirilemez.

COMMON PROGRAMMING ERROR



**Soyut bir sınıfı örneklendirmeye çalışmak
derleme hatasına yol açacaktır.**



- ▶ Çok-biçimlilik, özellikle “katmanlı yazılım sistemlerinin” uygulanmasında etkilidir.
- ▶ Örnek: İşletim sistemleri ve aygıt sürücüleri;
 - ▶ Cihazlardan/cihazlara veri okuma veya yazma komutları belirli bir tekdüzeliğe sahip olabilir.
 - ▶ Aygıt sürücüleri, işletim sistemi ile aygıtlar arasındaki tüm iletişimi denetler.
 - ▶ Bir aygıt sürücüsü nesnesine gönderilen bir yazma mesajı, o sürücü bağlamında ve belirli bir türdeki aygıtları nasıl yönettiğine göre yorumlanır.
 - ▶ Yazma çağrısının kendisi, sistemdeki herhangi bir başka aygıtta yazma işleminden gerçekten farklı değildir; bu aygıtta bellekten belirli sayıda bayt yerleştirin.
- ▶ Sürekli yeni cihazlar geliştirilmektedir.
 - ▶ Yeni bir cihaz satın aldığınızda, cihaz satıcısı tarafından sağlanan bir cihaz sürücüsü ile birlikte gelir ve siz onu bağlayıp sürücüyü kurduktan hemen sonra çalışır hale gelir.
 - ▶ Bu, Çok-biçimli sistemleri nasıl genişletilebilir hale getirdiğinin bir başka zarif örneğidir.



- ▶ Bir çalışan tarafından yönetilen miras hiyerarşisinin türüne göre bordro hesaplamaları yapmak için soyut bir yöntem ve çok-biçimlilik kullanın.
- ▶ Çalışan kalıtım hiyerarşisi gereksinimleri:
 - ▶ Bir şirket çalışanlarına haftalık olarak ödeme yapar.
 - ▶ Çalışanlar dört türdür:
 - ▶ Maaşlı çalışanlar; çalışılan saat sayısına bakılmaksızın sabit bir haftalık maaş ödenir.
 - ▶ Saatlik çalışanlar; saat başına ödenir ve 40 saatten fazla çalışılan saatler fazla mesai ücreti alırlar.
 - ▶ Komisyon çalışanları; satışlarının bir yüzdesi ödenir.
 - ▶ Taban maaşlı komisyon çalışanları: taban maaş artı satışlarının bir yüzdesi verilir.
- ▶ Cari ödeme dönemi için şirket, taban ücretli komisyon çalışanlarını taban maaşlarına %10 ekleyerek ödüllendirmeye karar verdi.
- ▶ Firma sizden bordro hesaplamalarını çok-biçimli olarak yapan bir Java uygulaması yazmanızı istiyor.



- Bir soyut “Employee” sınıfı olmalı
- SalariedEmployee, CommissionEmployee, HourlyEmployee alt sınıflar
- BasePlusCommissionEmployee ise CommissionEmployee ‘nin bir alt sınıfı

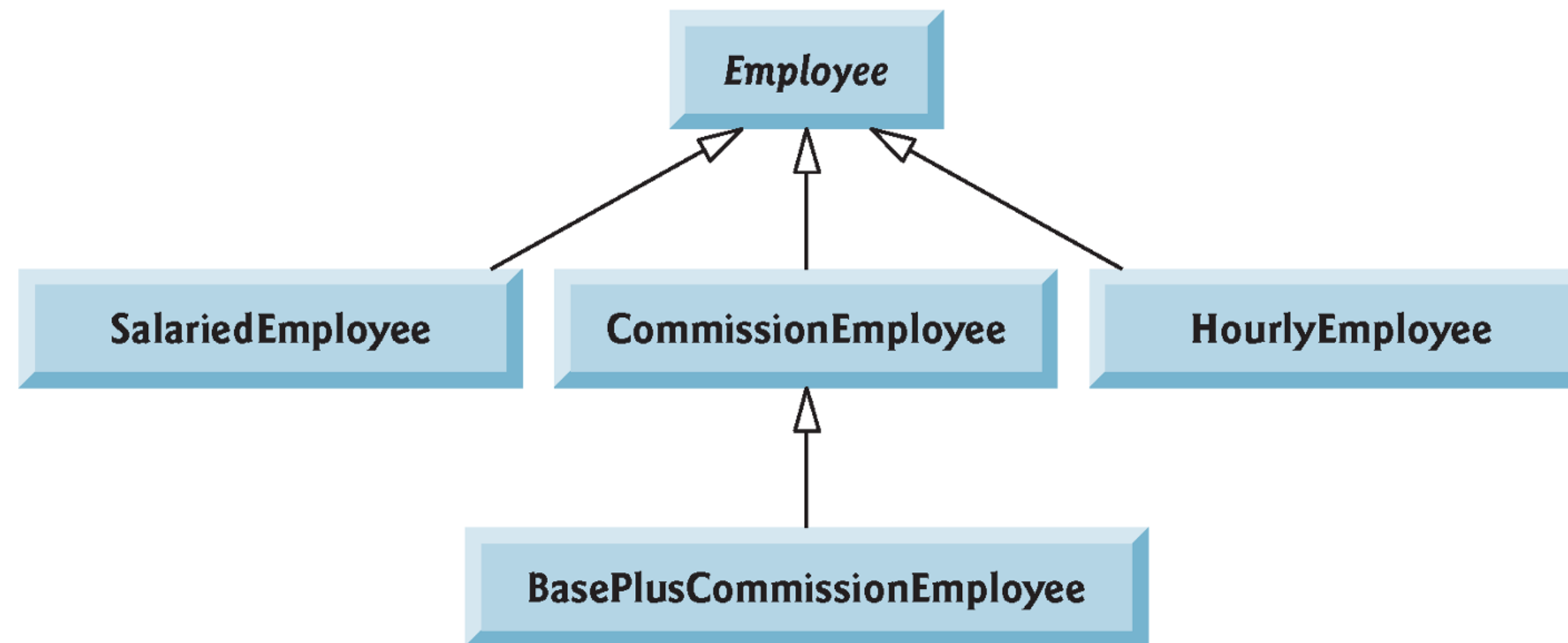


Fig. 10.2 | Employee hierarchy UML class diagram.



- ▶ “Employee” soyut sınıfı olarak, hiyerarşinin “arayüzünü”, yani bir programın tüm Employee nesnelerinde çağırabileceği yöntemler kümesini bildirir.
- ▶ Burada “arayüz” terimini, programların herhangi bir Employee alt sınıfının nesneleriyle iletişim kurabileceği çeşitli yollara atıfta bulunmak için genel anlamda kullanıyoruz.
- ▶ Her çalışanın soyut üst sınıf Employee’de tanımlanan bir adı, soyadı ve sosyal güvenlik numarası vardır.
- ▶ Class Employee Employee’nin örnek değişkenlerini değiştiren get() ve set() metotlarını ek olarak earnings() ve toString() metotlarını da sağlar.
- ▶ Tüm çalışanlar için bir earnings() geçerlidir, ancak her bir kazanç hesaplaması çalışanın sınıfına bağlıdır.
- ▶ Soyut bir metot—hangi miktarda kazancın geri dönmesi gerektiğini belirlemek için yeterli bilgi yok.
- ▶ Her alt sınıf, uygun bir uygulama ile earnings() metodunu override eder.
- ▶ Metot çağrıları çok-biçimli olarak işlenir.



	earnings	toString
Employee	abstract	<i>firstName lastName</i> social security number: <i>SSN</i>
Salaried-Employee	weeklySalary	salaried employee: <i>firstName lastName</i> social security number: <i>SSN</i> weekly salary: <i>weeklySalary</i>
Hourly-Employee	<pre>if (hours <= 40) { wage * hours } else if (hours > 40) { 40 * wage + (hours - 40) * wage * 1.5 }</pre>	hourly employee: <i>firstName lastName</i> social security number: <i>SSN</i> hourly wage: <i>wage</i> ; hours worked: <i>hours</i>
Commission-Employee	$\text{commissionRate} * \text{grossSales}$	commission employee: <i>firstName lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i>
BasePlus-Commission-Employee	$(\text{commissionRate} * \text{grossSales}) + \text{baseSalary}$	base salaried commission employee: <i>firstName lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i> ; base salary: <i>baseSalary</i>

Fig. 10.3 | Polymorphic interface for the Employee hierarchy classes.



```
1 // Fig. 10.4: Employee.java
2 // Employee abstract superclass.
3
4 public abstract class Employee {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8
9     // constructor
10    public Employee(String firstName, String lastName,
11                    String socialSecurityNumber) {
12        this.firstName = firstName;
13        this.lastName = lastName;
14        this.socialSecurityNumber = socialSecurityNumber;
15    }
16
```

Fig. 10.4 | Employee abstract superclass. (Part 1 of 2.)



```
17 // return first name
18 public String getFirstName() {return firstName;}
19
20 // return last name
21 public String getLastName() {return lastName;}
22
23 // return social security number
24 public String getSocialSecurityNumber() {return socialSecurityNumber;}
25
26 // return String representation of Employee object
27 @Override
28 public String toString() {
29     return String.format("%s %s\nsocial security number: %s",
30         getFirstName(), getLastName(), getSocialSecurityNumber());
31 }
32
33 // abstract method must be overridden by concrete subclasses
34 public abstract double earnings(); // no implementation here
35 }
```

Fig. 10.4 | Employee abstract superclass. (Part 2 of 2.)



```
1 // Fig. 10.5: SalariedEmployee.java
2 // SalariedEmployee concrete class extends abstract class Employee.
3
4 public class SalariedEmployee extends Employee {
5     private double weeklySalary;
6
7     // constructor
8     public SalariedEmployee(String firstName, String lastName,
9         String socialSecurityNumber, double weeklySalary) {
10         super(firstName, lastName, socialSecurityNumber);
11
12         if (weeklySalary < 0.0) {
13             throw new IllegalArgumentException(
14                 "Weekly salary must be >= 0.0");
15         }
16
17         this.weeklySalary = weeklySalary;
18     }
```

Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee. (Part 1 of 3.)



```
19
20 // set salary
21 public void setWeeklySalary(double weeklySalary) {
22     if (weeklySalary < 0.0) {
23         throw new IllegalArgumentException(
24             "Weekly salary must be >= 0.0");
25     }
26
27     this.weeklySalary = weeklySalary;
28 }
29
30 // return salary
31 public double getWeeklySalary() {return weeklySalary;}
32
```

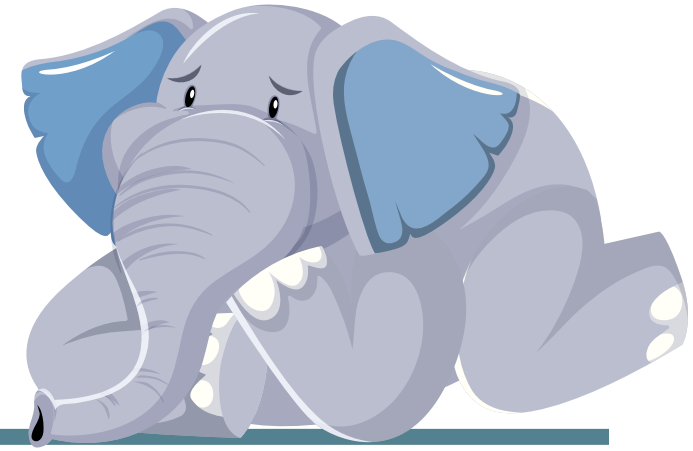
Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee. (Part 2 of 3.)



```
19
20 // set salary
21 public void setWeeklySalary(double weeklySalary) {
22     if (weeklySalary < 0.0) {
23         throw new IllegalArgumentException(
24             "Weekly salary must be >= 0.0");
25     }
26
27     this.weeklySalary = weeklySalary;
28 }
29
30 // return salary
31 public double getWeeklySalary() {return weeklySalary;}
32
33 // calculate earnings; override abstract method earnings in Employee
34 @Override
35 public double earnings() {return getWeeklySalary();}
36
37 // return String representation of SalariedEmployee object
38 @Override
39 public String toString() {
40     return String.format("salaried employee: %s%n%s: $%,.2f",
41         super.toString(), "weekly salary", getWeeklySalary());
42 }
43 }
```

Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee. (Part 3 of 3.)

ERROR PREVENTION TIP



Bir sınıfın kurucularından örnek metotları çağırmamanız gerektiğini söylemiştik!

Statik sınıf metotlarını çağırabilir ve üst sınıfın kurucularından birine gerekli çağrıyı yapabilirsiniz.

Bu tavsiyeye uyarsanız, sınıfın override metotlarını doğrudan veya dolaylı olarak çağırma probleminden kaçınırsınız, (çalışma zamanı hatalarına neden olabilir).



```
1 // Fig. 10.6: HourlyEmployee.java
2 // HourlyEmployee class extends Employee.
3
4 public class HourlyEmployee extends Employee {
5     private double wage; // wage per hour
6     private double hours; // hours worked for week
7
8     // constructor
9     public HourlyEmployee(String firstName, String lastName,
10        String socialSecurityNumber, double wage, double hours) {
11         super(firstName, lastName, socialSecurityNumber);
12
13         if (wage < 0.0) { // validate wage
14             throw new IllegalArgumentException("Hourly wage must be >= 0.0");
15         }
16
17         if ((hours < 0.0) || (hours > 168.0)) { // validate hours
18             throw new IllegalArgumentException(
19                 "Hours worked must be >= 0.0 and <= 168.0");
20         }
21
22         this.wage = wage;
23         this.hours = hours;
24     }
}
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 2 of 5.)



```
25
26 // set wage
27 public void setWage(double wage) {
28     if (wage < 0.0) { // validate wage
29         throw new IllegalArgumentException("Hourly wage must be >= 0.0");
30     }
31
32     this.wage = wage;
33 }
34
35 // return wage
36 public double getWage() {return wage;}
37
38 // set hours worked
39 public void setHours(double hours) {
40     if ((hours < 0.0) || (hours > 168.0)) { // validate hours
41         throw new IllegalArgumentException(
42             "Hours worked must be >= 0.0 and <= 168.0");
43     }
44
45     this.hours = hours;
46 }
47
48 // return hours worked
49 public double getHours() {return hours;}
50
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 4 of 5.)



```
51 // calculate earnings; override abstract method earnings in Employee
52 @Override
53 public double earnings() {
54     if (getHours() <= 40) { // no overtime
55         return getWage() * getHours();
56     }
57     else {
58         return 40 * getWage() + (getHours() - 40) * getWage() * 1.5;
59     }
60 }
61
62 // return String representation of HourlyEmployee object
63 @Override
64 public String toString() {
65     return String.format("hourly employee: %s\n%s: $%,.2f; %s: $%,.2f",
66         super.toString(), "hourly wage", getWage(),
67         "hours worked", getHours());
68 }
69 }
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 5 of 5.)



```
1 // Fig. 10.7: CommissionEmployee.java
2 // CommissionEmployee class extends Employee.
3
4 public class CommissionEmployee extends Employee {
5     private double grossSales; // gross weekly sales
6     private double commissionRate; // commission percentage
7 }
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 1 of 5.)



```
8      // constructor
9      public CommissionEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double grossSales,
11         double commissionRate) {
12         super(firstName, lastName, socialSecurityNumber);
13
14         if (commissionRate <= 0.0 || commissionRate >= 1.0) { // validate
15             throw new IllegalArgumentException(
16                 "Commission rate must be > 0.0 and < 1.0");
17         }
18
19         if (grossSales < 0.0) { // validate
20             throw new IllegalArgumentException("Gross sales must be >= 0.0");
21         }
22
23         this.grossSales = grossSales;
24         this.commissionRate = commissionRate;
25     }
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 2 of 5.)



```
26
27 // set gross sales amount
28 public void setGrossSales(double grossSales) {
29     if (grossSales < 0.0) { // validate
30         throw new IllegalArgumentException("Gross sales must be >= 0.0");
31     }
32
33     this.grossSales = grossSales;
34 }
35
36 // return gross sales amount
37 public double getGrossSales() {return grossSales;}
38
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 3 of 5.)



```
39 // set commission rate
40 public void setCommissionRate(double commissionRate) {
41     if (commissionRate <= 0.0 || commissionRate >= 1.0) { // validate
42         throw new IllegalArgumentException(
43             "Commission rate must be > 0.0 and < 1.0");
44     }
45
46     this.commissionRate = commissionRate;
47 }
48
49 // return commission rate
50 public double getCommissionRate() {return commissionRate;}
51
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 4 of 5.)



```
52 // calculate earnings; override abstract method earnings in Employee
53 @Override
54 public double earnings() {
55     return getCommissionRate() * getGrossSales();
56 }
57
58 // return String representation of CommissionEmployee object
59 @Override
60 public String toString() {
61     return String.format("%s: %s%n%s: $%,.2f; %s: %.2f",
62         "commission employee", super.toString(),
63         "gross sales", getGrossSales(),
64         "commission rate", getCommissionRate());
65 }
66 }
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 5 of 5.)



```
1 // Fig. 10.8: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class extends CommissionEmployee.
3
4 public class BasePlusCommissionEmployee extends CommissionEmployee {
5     private double baseSalary; // base salary per week
6
7     // constructor
8     public BasePlusCommissionEmployee(String firstName, String lastName,
9         String socialSecurityNumber, double grossSales,
10        double commissionRate, double baseSalary) {
11        super(firstName, lastName, socialSecurityNumber,
12            grossSales, commissionRate);
13
14        if (baseSalary < 0.0) { // validate baseSalary
15            throw new IllegalArgumentException("Base salary must be >= 0.0");
16        }
17
18        this.baseSalary = baseSalary;
19    }
```

Fig. 10.8 | BasePlusCommissionEmployee class extends CommissionEmployee. (Part 1 of 3.)



```
20
21 // set base salary
22 public void setBaseSalary(double baseSalary) {
23     if (baseSalary < 0.0) { // validate baseSalary
24         throw new IllegalArgumentException("Base salary must be >= 0.0");
25     }
26
27     this.baseSalary = baseSalary;
28 }
29
30 // return base salary
31 public double getBaseSalary() {return baseSalary;}
32
```

Fig. 10.8 | BasePlusCommissionEmployee class extends CommissionEmployee. (Part 2 of 3.)



```
33 // calculate earnings; override method earnings in CommissionEmployee
34 @Override
35 public double earnings() {return getBaseSalary() + super.earnings();}
36
37 // return String representation of BasePlusCommissionEmployee object
38 @Override
39 public String toString() {
40     return String.format("%s %s; %s: $%,.2f",
41         "base-salaried", super.toString(),
42         "base salary", getBaseSalary());
43 }
44 }
```

Fig. 10.8 | BasePlusCommissionEmployee class extends CommissionEmployee. (Part 3 of 3.)



- ▶ Şekil 10.9, dört concrete sınıfın her birinin bir nesnesini oluşturur.
- ▶ Bu nesneleri, her nesnenin kendi türündeki değişkenler aracılığıyla çok-biçimli olmayan bir şekilde düzenler.
- ▶ Ardından bir Employee değişkenleri dizisini kullanarak çok-biçimli olarak işler.
- ▶ Nesneleri çok-biçimli olarak işlerken, program her BasePlusCommissionEmployee için taban maaşını %10 artırır.
- ▶ Yürütme zamanında nesnenin türünün belirlenmesini gerektirir.
- ▶ Son olarak program, Employee dizisindeki her nesnenin türünü çok-biçimli olarak belirler ve çıktısını verir.



```
1 // Fig. 10.9: PayrollSystemTest.java
2 // Employee hierarchy test program.
3
4 public class PayrollSystemTest {
5     public static void main(String[] args) {
6         // create subclass objects
7         SalariedEmployee salariedEmployee =
8             new SalariedEmployee("John", "Smith", "111-11-1111", 800.00);
9         HourlyEmployee hourlyEmployee =
10            new HourlyEmployee("Karen", "Price", "222-22-2222", 16.75, 40);
11        CommissionEmployee commissionEmployee =
12            new CommissionEmployee(
13                "Sue", "Jones", "333-33-3333", 10000, .06);
14        BasePlusCommissionEmployee basePlusCommissionEmployee =
15            new BasePlusCommissionEmployee(
16                "Bob", "Lewis", "444-44-4444", 5000, .04, 300);
17    }
```

Fig. 10.9 | Employee hierarchy test program. (Part 1 of 8.)



```
18      System.out.println("Employees processed individually:");
19
20      System.out.printf("%n%s%n%s: $%,.2f%n%n",
21          salariedEmployee, "earned", salariedEmployee.earnings());
22      System.out.printf("%s%n%s: $%,.2f%n%n",
23          hourlyEmployee, "earned", hourlyEmployee.earnings());
24      System.out.printf("%s%n%s: $%,.2f%n%n",
25          commissionEmployee, "earned", commissionEmployee.earnings());
26      System.out.printf("%s%n%s: $%,.2f%n%n",
27          basePlusCommissionEmployee,
28          "earned", basePlusCommissionEmployee.earnings());
29
```

Fig. 10.9 | Employee hierarchy test program. (Part 2 of 8.)



```
30      // create four-element Employee array
31      Employee[] employees = new Employee[4];
32
33      // initialize array with Employees
34      employees[0] = salariedEmployee;
35      employees[1] = hourlyEmployee;
36      employees[2] = commissionEmployee;
37      employees[3] = basePlusCommissionEmployee;
38
39      System.out.printf("Employees processed polymorphically:%n%n");
40
```

Fig. 10.9 | Employee hierarchy test program. (Part 3 of 8.)



```
41 // generically process each element in array employees
42 for (Employee currentEmployee : employees) {
43     System.out.println(currentEmployee); // invokes toString
44
45     // determine whether element is a BasePlusCommissionEmployee
46     if (currentEmployee instanceof BasePlusCommissionEmployee) {
47         // downcast Employee reference to
48         // BasePlusCommissionEmployee reference
49         BasePlusCommissionEmployee employee =
50             (BasePlusCommissionEmployee) currentEmployee;
51
52         employee.setBaseSalary(1.10 * employee.getBaseSalary());
53
54         System.out.printf(
55             "new base salary with 10%% increase is: $%,.2f%n",
56             employee.getBaseSalary());
57     }
58
59     System.out.printf(
60         "earned $%,.2f%n%n", currentEmployee.earnings());
61 }
```

Fig. 10.9 | Employee hierarchy test program. (Part 4 of 8.)



```
62
63     // get type name of each object in employees array
64     for (int j = 0; j < employees.length; j++) {
65         System.out.printf("Employee %d is a %s\n", j,
66             employees[j].getClass().getName());
67     }
68 }
69 }
```

Fig. 10.9 | Employee hierarchy test program. (Part 5 of 8.)



Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: \$800.00
earned: \$800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: \$16.75; hours worked: 40.00
earned: \$670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: \$10,000.00; commission rate: 0.06
earned: \$600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: \$5,000.00; commission rate: 0.04; base salary: \$300.00
earned: \$500.00

Fig. 10.9 | Employee hierarchy test program. (Part 6 of 8.)



Employees processed polymorphically:

salariied employee: John Smith

social security number: 111-11-1111

weekly salary: \$800.00

earned \$800.00

hourly employee: Karen Price

social security number: 222-22-2222

hourly wage: \$16.75; hours worked: 40.00

earned \$670.00

commission employee: Sue Jones

social security number: 333-33-3333

gross sales: \$10,000.00; commission rate: 0.06

earned \$600.00

base-salariied commission employee: Bob Lewis

social security number: 444-44-4444

gross sales: \$5,000.00; commission rate: 0.04; base salary: \$300.00

new base salary with 10% increase is: \$330.00

earned \$530.00

Fig. 10.9 | Employee hierarchy test program. (Part 7 of 8.)



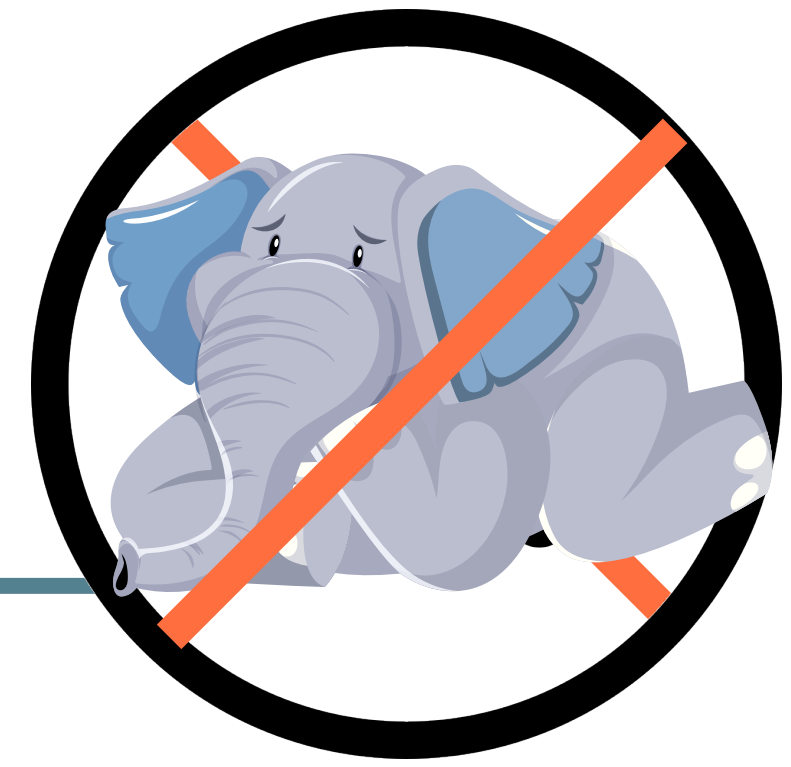
```
Employee 0 is a SalariedEmployee  
Employee 1 is a HourlyEmployee  
Employee 2 is a CommissionEmployee  
Employee 3 is a BasePlusCommissionEmployee
```

Fig. 10.9 | Employee hierarchy test program. (Part 8 of 8.)



- ▶ `toString()` ve `earnings()` metotlarına yapılan tüm çağrılar, `currentEmployee`'nin başvurduğu nesnenin türüne göre yürütme zamanında çözülür.
- ▶ Dinamik bağlama veya geç bağlama olarak bilinir.
- ▶ Java, derleme zamanında değil, yürütme zamanında hangi sınıfın `toString` metodunun çağrılacağına karar verir.
- ▶ Yalnızca üst sınıfın metotlarını çağırmak için bir üst sınıf başvurusu kullanılabilir; alt sınıf metodu uygulamaları çok-biçimli olarak çağrılır.
- ▶ Yalnızca bir alt sınıf metodunu doğrudan bir üst sınıf başvurusunda çağırmaya çalışmak bir derleme hatasıdır.

COMMON PROGRAMMING ERROR



Üst sınıf değişkeni alt sınıf değişkenine atamak derleme hatasıdır.

“is-a” ilişkisi olmaya sınıflar arasında downcasting yapılırsa “ClassCastException” istisnası ortaya çıkar.

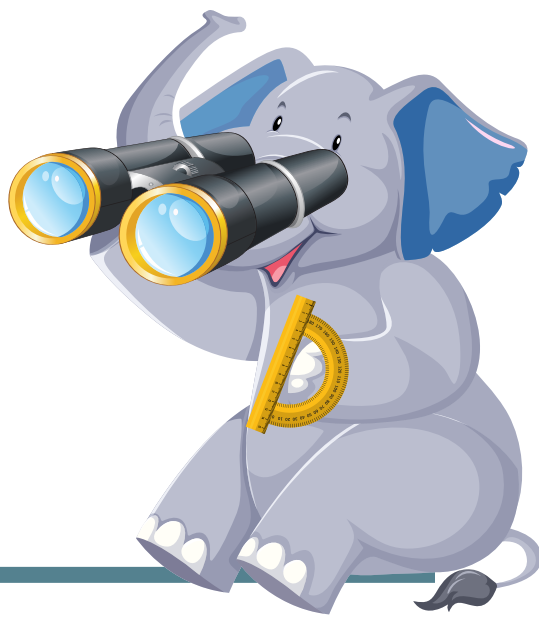


- ▶ Her nesne kendi sınıfını bilir ve bu bilgilere tüm sınıfların Object sınıfından miras aldığı `getClass()` metodu aracılığıyla erişebilir.
- ▶ `getClass` metodu, sınıf adı da dahil olmak üzere nesnenin türü hakkında bilgi içeren `Class` türünden (`java.lang` paketinden) bir nesne döndürür.
- ▶ `getClass` çağrısının sonucu, nesnenin sınıf adını almak için `getName()` çağırmak için kullanılır.
- ▶ **HATIRLATMA:** Bu metodlar `java.lang.toString()` metodunu içinde kullanılıyor. (Override edilmemişse.)



- ▶ Bir üst sınıftaki final bir metot, bir alt sınıfta geçersiz kılınamaz (override).
- ▶ Private olarak bildirilen metotlar da, bir alt sınıfta geçersiz kılmak mümkün olmadığından, örtük olarak final olacaktır.
- ▶ Static olarak bildirilen metotlar örtük olarak final olacaktır.
- ▶ Final metodun bildirimi asla değişmez, bu nedenle tüm alt sınıflar aynı metot uygulamasını kullanır ve final metotlara yapılan çağrılar derleme zamanında çözülür;
- ▶ Bu statik bağlama (static binding) olarak bilinir.
- ▶ **HATIRLATMA:** Örnekteki earnings() metoduna yapılan tüm çağrılar, currentEmployee'nin başvurduğu nesnenin türüne göre yürütme zamanında çözülür. (Dynamic binding)

SOFTWARE ENGINEERING OBSERVATION



Binding: Bir metod bildirimi ile çağrısı arasındaki link.

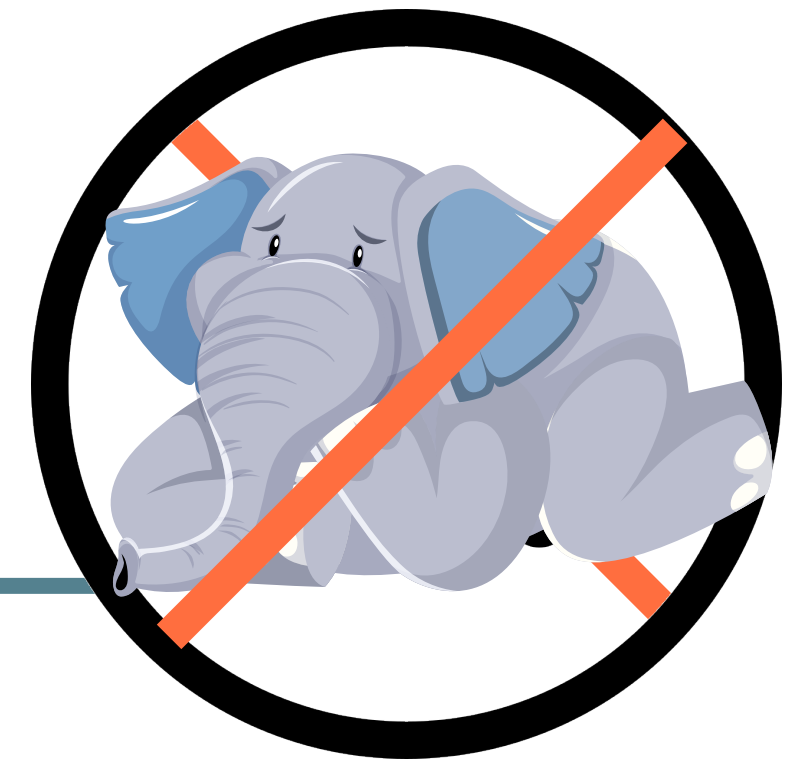
Static b (overloading), dynamic b (overriding) örnek olabilir.

Static derleme zamanı, dynamic çalışma zamanı çözülür.



- ▶ Bir final sınıf, bir alt sınıf oluşturmak için genişletilemez.
- ▶ Bir final sınıfındaki tüm metotlar örtük olarak final olur.
- ▶ Class String, bir final sınıf örneğidir.
- ▶ String'in bir alt sınıfını oluşturmanıza izin verilseydi, o alt sınıfın nesneleri, String'lerin beklendiği her yerde kullanılabilirdi.
- ▶ String sınıfı genişletilemediğinden, String kullanan programlar Java API'sinde belirtildiği gibi String nesnelerinin işlevselliğine güvenebilir.
- ▶ Sınıfı final yapmak, programcıların güvenlik kısıtlamalarını aşabilecek alt sınıflar oluşturmasını da önler.

COMMON PROGRAMMING ERROR



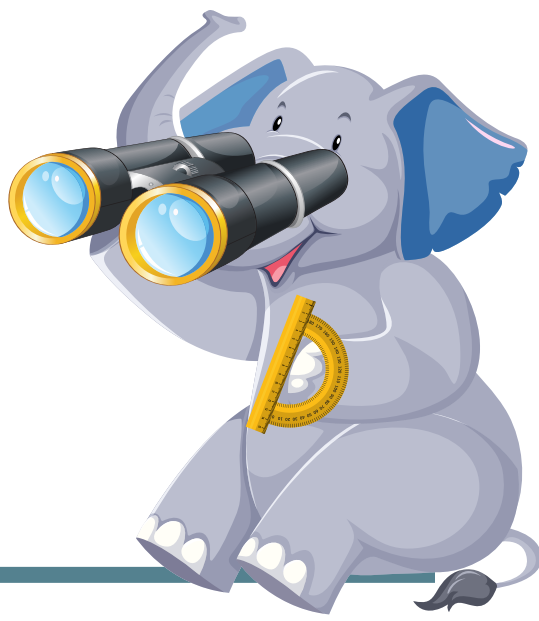
Final bir sınıftan miras alan bir alt sınıf bildirmek derleme hatasına yol açar.

Final sınıflar composition olarak kullanılabilir.



- ▶ Constructor içinden override edilebilir metotları çağırmayın.
- ▶ Bir alt sınıf nesnesi oluştururken, bu, alt sınıf nesnesi tamamen başlatılmadan önce override edilen bir metodun çağırılmasına neden olabilir.
- ▶ Bir alt sınıf nesnesi oluşturduğunuzda, Constructor önce doğrudan üst sınıfın Constructor lardan birini çağırıldığını hatırlayın.
- ▶ Bir nesne üretilecektir. (Alt sınıfta bir override metot var.)
- ▶ Önce süper Constructor çağırıldı.
- ▶ Üst sınıfta da aynı metot bulunmaktadır.
- ▶ Dolayısıyla alt sınıf constructor metodu çağırmadan, üst sınıf çağırabilir.
- ▶ Çağrılan alt sınıf metodu, alt sınıf constructor gövdesinde henüz gerçekleştirilmemiş başlatmaya bağlıysa, bu, anlaşılması güç, algılanması zor hatalara yol açabilir.
- ▶ Bir constructor dan statik bir metot çağırarak kabul edilebilir.

SOFTWARE ENGINEERING OBSERVATION



Java API sınıfların çoğunu final olarak bildirmez.

Böylelikle kalıtım ve çok-biçimlilik mümkün oluyor.

Sadece belirli güvenlik hususlarından dolayı final sınıf tanımlanmıştır.



- ▶ Bir sonraki örneğimiz, önceki bölümdeki bordro sistemini yeniden inceliyor.
- ▶ İlgili şirketin, tek bir borç hesabı uygulamasında birkaç muhasebe işlemi gerçekleştirmek istediğini varsayalım.
 - ▶ Her çalışana ödenmesi gereken kazancın hesaplanması
 - ▶ Faturalarının her biri için gerekli ödemeyi hesaplayın (yani, satın alınan mallar için faturalar)
- ▶ Her iki işlem de bir tür ödeme tutarı elde etmekle ilgilidir.
 - ▶ Bir çalışan için ödeme, çalışanın kazancını ifade eder.
 - ▶ Bir fatura için ödeme, faturada listelenen malların toplam maliyetini ifade eder.



- ▶ Arayüzler (interfaces), ilgisiz sınıfların bir dizi ortak yöntem uygulamasını gerektiren bir yetenek sunar.
- ▶ Arayüzler, insanlar ve sistemler gibi şeylerin birbirleriyle etkileşim kurma yollarını tanımlar ve standartlaştırır.
- ▶ Örnek: Bir telsizdeki kontroller, telsiz kullanıcıları ile telsizin dahili bileşenleri arasında bir arayüz görevi görür.
- ▶ Yalnızca sınırlı sayıda işlem gerçekleştirebilir (örneğin, istasyonu değiştirin, ses seviyesini ayarlayın, AM ve FM arasında seçim yapın)
- ▶ Farklı telsizler, kontrolleri farklı şekillerde uygulayabilir (örneğin, basmalı düğmeler, kadranlar, sesli komutlar kullanarak).
- ▶ Arayüz, bir telsizin hangi işlemleri gerçekleştirmesine izin vermesi gerektiğini belirtir, ancak işlemlerin nasıl gerçekleştirileceğini belirtmez.

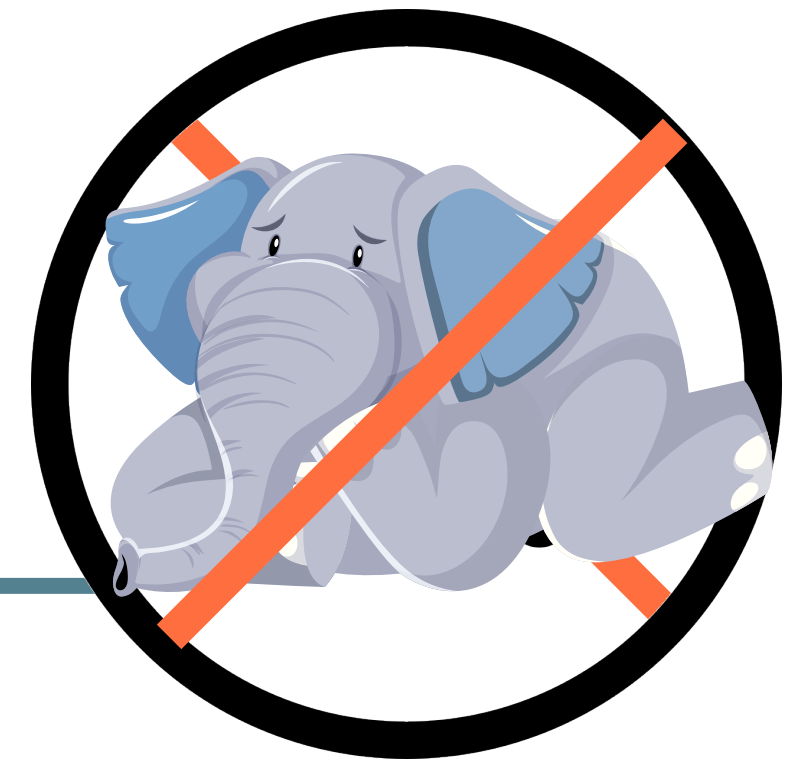


- ▶ **Java interface: bir nesne üzerinde çağrılacak bir dizi yöntemi tanımlar.**
- ▶ **Bir interface bildirimi, “interface” anahtar sözcüğüyle başlar**
- ▶ **Yalnızca sabitleri ve soyut metotları içerir.**
- ▶ **Tüm arayüz üyeleri “public” olmalıdır.**
- ▶ **Somut metot bildirimleri ve örnek değişkenler gibi herhangi bir uygulama detayını belirtmez.**
- ▶ **Bir interface içinde bildirilen tüm yöntemler, dolaylı olarak “public abstract” metotlardır.**
- ▶ **Tüm alanlar dolaylı olarak: public static ve final olur.**



- ▶ Bir interface kullanmak için somut bir sınıf, “implements” ile onu belirtmeli.
- ▶ Interface içinde tüm metotları belirtilen imzayla bildirmelidir.
- ▶ “implements” anahtar sözcüğünü ve “interface adı” nı sınıf bildiriminizin ilk satırının sonuna ekleyin.
- ▶ Interface tüm metotlarını uygulamayan bir sınıf, soyut bir sınıftır ve “abstract” olarak bildirilmelidir.
- ▶ Bir Interface kullanmak, derleyici ile "Interface tarafından belirtilen tüm metotları bildireceğim veya sınıfımı “abstract” olarak bildireceğim" şeklinde bir sözleşme imzalamaya benzer.

COMMON PROGRAMMING ERROR



Bir interface implements eden somut bir sınıfta, her hangi bir abstract metodu implement etmemek derleme hatasına yol açar. Çözüm için 2 seçenek var.

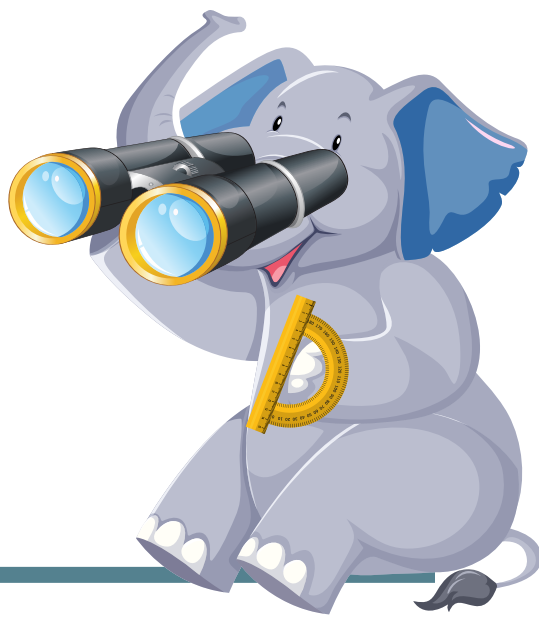
1— Tüm hepsini implement et.

2— Somut sınıfı abstract yap.



- ▶ Bir interface genellikle farklı sınıfların (yani ilgisiz sınıfların) ortak yöntemleri ve sabitleri paylaşması gerektiğinde kullanılır.
- ▶ Aynı yöntem çağrılarına yanıt vererek ilgisiz sınıfların nesnelerinin ÇOK-BİÇİMLİ olarak işlenmesine izin verir.
- ▶ İstenen işlevi açıklayan bir interface oluşturabilir, ardından bu interface, bu işlevi gerektiren herhangi bir sınıfa uygulayabilirsiniz.
- ▶ Miras alınan bir implement işlemi olmadığında, yani hiçbir alan ve hiçbir somut metot implement işlemi olmadığında, soyut bir sınıf yerine bir interface kullanılmalıdır.
- ▶ public abstract sınıf gibi, interface de tipik olarak public türlerdir.
- ▶ Bir public interface, interface ile aynı ada ve .java dosya adı uzantısına sahip bir dosyada bildirilmelidir.

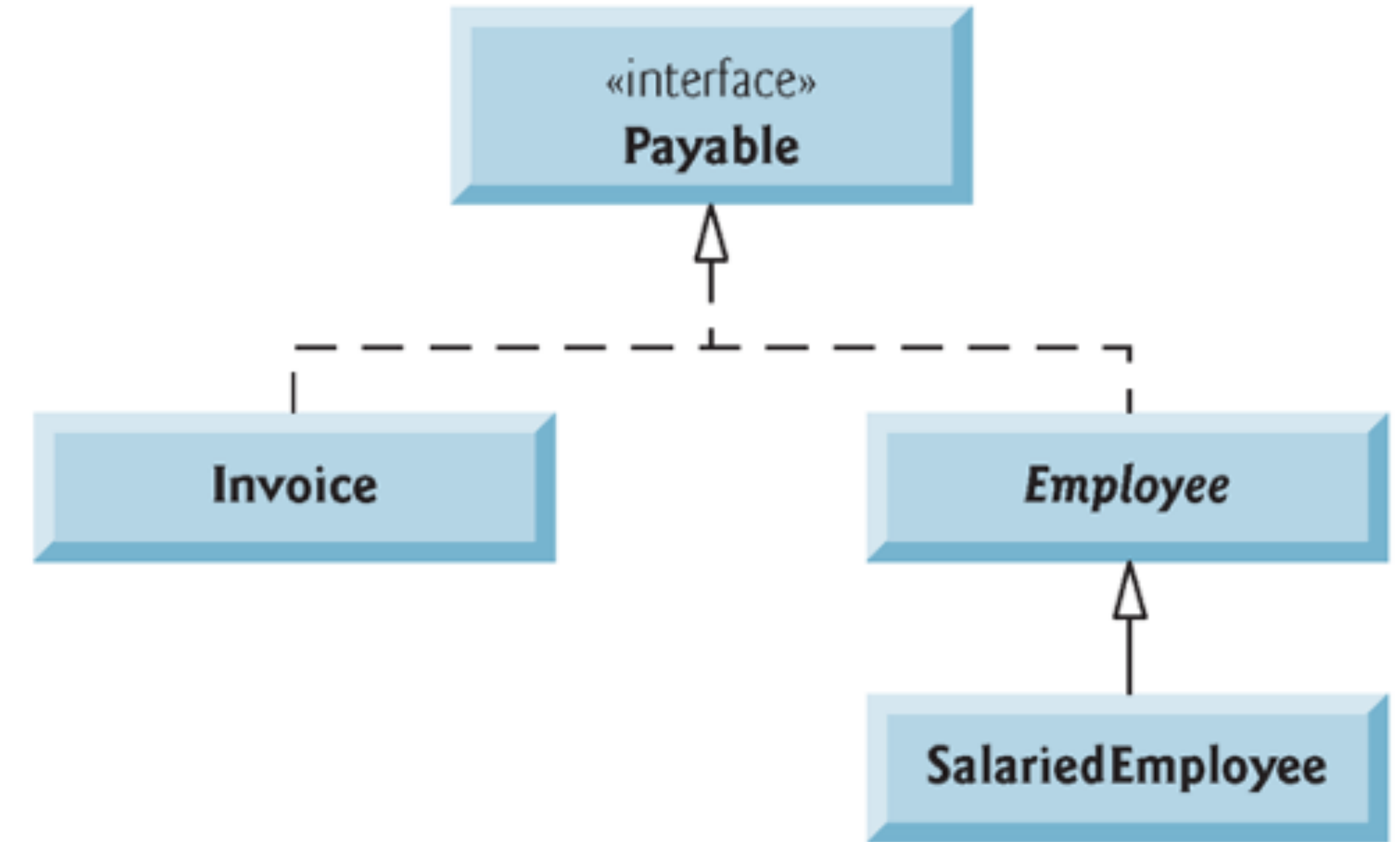
SOFTWARE ENGINEERING OBSERVATION



JAVA 8 özellikleri ile birlikte çoğu geliştirici yeni teknolojileri modelleme için interface lerin sınıflardan daha önemli olduğunu düşünmektedir.



- Sonraki örnek, çalışanlar ve faturalar için ödemeleri belirleyebilen bir uygulama oluşturur.
- Invoice ve Employee sınıflarının ikisi de şirketin bir ödeme tutarı hesaplayabilmesi gereken şeyleri temsil eder.
- Her iki sınıf da Payable interface ini implement eder. Böylece bir program hem Invoice nesnelerinde hem de Employee nesnelerinde getPaymentAmount metodunu çağırabilir.
- Faturaların ve Çalışanların çok-biçimli olarak işlenmesini sağlar.



Payable hierarchy UML class diagram.



```
1 // Fig. 10.11: Payable.java
2 // Payable interface declaration.
3
4 public interface Payable {
5     public abstract double getPaymentAmount(); // no implementation
6 }
```

Fig. 10.11 | Payable interface declaration.



- ▶ Invoice sınıfı, yalnızca tek bir tür fatura bilgilerini içeren basit bir faturayı temsil eder.
- ▶ Java, alt sınıfların birden fazla üst sınıftan miras almasına izin vermez, ancak ihtiyaç duyduğu kadar çok interface implement etmesine izin verir.
- ▶ Birden fazlasını uygulamak için, sınıf bildiriminde anahtar kelime uygulandıktan sonra virgülle ayrılmış bir interface adları listesi kullanın, örneğin:
- ▶ `public class ClassName extends SuperclassName, implements FirstInterface, SecondInterface,... { ... }`



```
1 // Fig. 10.12: Invoice.java
2 // Invoice class that implements Payable.
3
4 public class Invoice implements Payable {
5     private final String partNumber;
6     private final String partDescription;
7     private final int quantity;
8     private final double pricePerItem;
9 }
```

Fig. 10.12 | Invoice class that implements Payable. (Part 1 of 4.)



```
10    // constructor
11    public Invoice(String partNumber, String partDescription, int quantity,
12        double pricePerItem) {
13        if (quantity < 0) { // validate quantity
14            throw new IllegalArgumentException("Quantity must be >= 0");
15        }
16
17        if (pricePerItem < 0.0) { // validate pricePerItem
18            throw new IllegalArgumentException(
19                "Price per item must be >= 0");
20        }
21
22        this.quantity = quantity;
23        this.partNumber = partNumber;
24        this.partDescription = partDescription;
25        this.pricePerItem = pricePerItem;
26    }
```

Fig. 10.12 | Invoice class that implements Payable. (Part 2 of 4.)



```
27
28 // get part number
29 public String getPartNumber() {return partNumber;}
30
31 // get description
32 public String getPartDescription() {return partDescription;}
33
34 // get quantity
35 public int getQuantity() {return quantity;}
36
37 // get price per item
38 public double getPricePerItem() {return pricePerItem;}
39
```

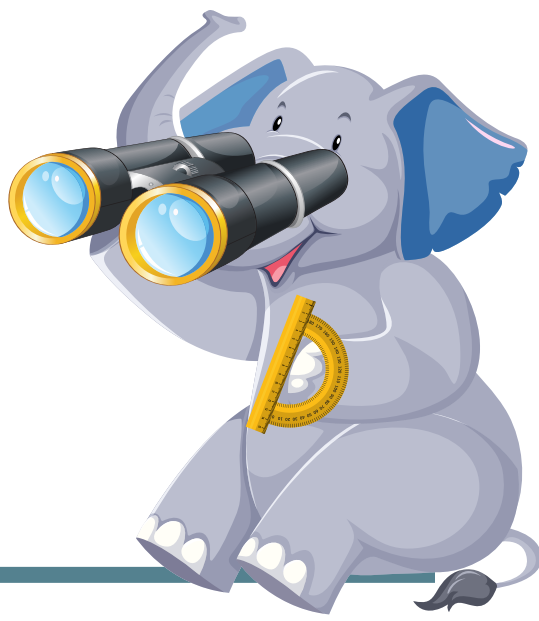
Fig. 10.12 | Invoice class that implements Payable. (Part 3 of 4.)



```
40 // return String representation of Invoice object
41 @Override
42 public String toString() {
43     return String.format("%s: %n%s: %s (%s) %n%s: %d %n%s: $%,.2f",
44         "invoice", "part number", getPartNumber(), getPartDescription(),
45         "quantity", getQuantity(), "price per item", getPricePerItem());
46 }
47
48 // method required to carry out contract with interface Payable
49 @Override
50 public double getPaymentAmount() {
51     return getQuantity() * getPricePerItem(); // calculate total cost
52 }
53 }
```

Fig. 10.12 | Invoice class that implements Payable. (Part 4 of 4.)

SOFTWARE ENGINEERING OBSERVATION



Çoklu interface implement eden bir sınıfın tüm nesneleri, implement edilen her bir interface tipi ile “is-a” ilişkisine sahiptir.



- ▶ Şimdi Payable interface implement etmek için Employee sınıfını değiştirelim.
- ▶ Bu sınıf bildirimi, iki istisna dışında, ile aynıdır:
 - ▶ Satır 4, Employee sınıfının artık Payable'i implement edildiğini gösterir.
 - ▶ Satır 38, Payable interface getPaymentAmount metodunu implement eder.
- ▶ getPaymentAmount, yalnızca Employee sınıfından, soyut metot earnings() çağırır.
- ▶ Yürütme zamanında, bir Employee alt sınıfının bir nesnesinde getPaymentAmount çağrıldığında, getPaymentAmount, o alt sınıfın, o alt sınıf türündeki nesneler için kazançların nasıl hesaplanacağını bilen somut earnings() metodunu çağırır.



```
1 // Fig. 10.13: Employee.java
2 // Employee abstract superclass that implements Payable.
3
4 public abstract class Employee implements Payable {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8
9     // constructor
10    public Employee(String firstName, String lastName,
11                    String socialSecurityNumber) {
12        this.firstName = firstName;
13        this.lastName = lastName;
14        this.socialSecurityNumber = socialSecurityNumber;
15    }
16
```

Fig. 10.13 | Employee abstract superclass that implements Payable. (Part 1 of 3.)



```
17 // return first name
18 public String getFirstName() {return firstName;}
19
20 // return last name
21 public String getLastName() {return lastName;}
22
23 // return social security number
24 public String getSocialSecurityNumber() {return socialSecurityNumber;}
25
26 // return String representation of Employee object
27 @Override
28 public String toString() {
29     return String.format("%s %s\nsocial security number: %s",
30         getFirstName(), getLastName(), getSocialSecurityNumber());
31 }
32
```

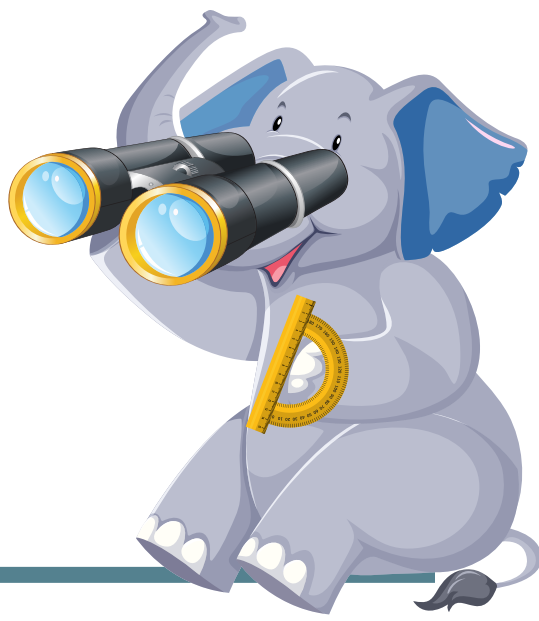
Fig. 10.13 | Employee abstract superclass that implements Payable. (Part 2 of 3.)



```
33 // abstract method must be overridden by concrete subclasses
34 public abstract double earnings(); // no implementation here
35
36 // implementing getPaymentAmount here enables the entire Employee
37 // class hierarchy to be used in an app that processes Payables
38 public double getPaymentAmount() {return earnings();}
39 }
```

Fig. 10.13 | Employee abstract superclass that implements Payable. (Part 3 of 3.)

SOFTWARE ENGINEERING OBSERVATION



Kalıtım ve arayüz kavramları “is-a” ilişkisi bağlamında benzer uygulamalara sahiptir.

Interface implement eden bir sınıf nesnesi interface tipinin bir nesnesi gibi düşünülebilir. Bu sınıfın alt sınıfları için de durum bu şekildedir.



- ▶ PayableInterfaceTest (Şekil 10.14), Payable interface için tek bir uygulamada bir dizi Invoice ve Employee nesnesini çok biçimli olarak işlemek için kullanılabileceğini göstermektedir.
- ▶ 18-23 satırları, her bir Payable nesnesini payableObjects içinde çok biçimli olarak işler ve her nesnenin Dize temsilini ve ödeme tutarını görüntüler.
- ▶ 21. satır, toString metodunu Payable interface referansı aracılığıyla çağırır, toString Payable interface içinde bildirilmemiştir. Ancak,
 - ▶ Tüm referanslar (interface türlerine ait olanlar dahil) Object'i extend eden nesneleri gösterir.
 - ▶ Dolayısıyla hepsi toString metoduna sahiptir.



```
1 // Fig. 10.14: PayableInterfaceTest.java
2 // Payable interface test program processing Invoices and
3 // Employees polymorphically.
4 public class PayableInterfaceTest {
5     public static void main(String[] args) {
6         // create four-element Payable array
7         Payable[] payableObjects = new Payable[] {
8             new Invoice("01234", "seat", 2, 375.00),
9             new Invoice("56789", "tire", 4, 79.95),
10            new SalariedEmployee("John", "Smith", "111-11-1111", 800.00),
11            new SalariedEmployee("Lisa", "Barnes", "888-88-8888", 1200.00)
12        };
13    }
```

Fig. 10.14 | Payable interface test program processing Invoices and Employees polymorphically. (Part I of 3.)



```
14      System.out.println(  
15          "Invoices and Employees processed polymorphically:");  
16  
17      // generically process each element in array payableObjects  
18      for (Payable currentPayable : payableObjects) {  
19          // output currentPayable and its appropriate payment amount  
20          System.out.printf("%n%s %npayment due: $%,.2f%n",  
21              currentPayable.toString(), // could invoke implicitly  
22              currentPayable.getPaymentAmount());  
23      }  
24  }  
25 }
```

Fig. 10.14 | Payable interface test program processing Invoices and Employees polymorphically. (Part 2 of 3.)



Invoices and Employees processed polymorphically:

invoice:
part number: 01234 (seat)
quantity: 2
price per item: \$375.00
payment due: \$750.00

invoice:
part number: 56789 (tire)
quantity: 4
price per item: \$79.95
payment due: \$319.80

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: \$800.00
payment due: \$800.00

salaried employee: Lisa Barnes
social security number: 888-88-8888
weekly salary: \$1,200.00
payment due: \$1,200.00

Fig. 10.14 | Payable interface test program processing Invoices and Employees polymorphically. (Part 3 of 3.)



► Java uygulamaları geliştirilirken interface oldukça sık kullanılır. Java API bir çok interface içerir.

Interface	Description
Comparable	Java contains several comparison operators (e.g., <, <=, >, >=, ==, !=) that allow you to compare primitive values. However, these operators <i>cannot</i> be used to compare objects. Interface Comparable is used to allow objects of a class that implements the interface to be compared to one another. Interface Comparable is commonly used for ordering objects in a collection such as an ArrayList. We use Comparable in Chapter 16, Generic Collections, and Chapter 20, Generic Classes and Methods: A Deeper Look.

Fig. 10.15 | Common interfaces of the Java API. (Part 1 of 4.)



► Java uygulamaları geliştirilirken interface oldukça sık kullanılır. Java API bir çok interface içerir.

Interface	Description
Comparable	
Serializable	An interface used to identify classes whose objects can be written to (i.e., serialized) or read from (i.e., deserialized) some type of storage (e.g., file on disk, database field) or transmitted across a network.
Runnable	Implemented by any class that represents a task to perform. Objects of such a class are often executed in parallel using a technique called <i>multithreading</i> (discussed in Chapter 23, Concurrency). The interface contains one method, <code>run</code> , which specifies the behavior of an object when executed.

Fig. 10.15 | Common interfaces of the Java API. (Part 2 of 4.)

Fig. 10.15 | Common interfaces of the Java API. (Part 2 of 4.)



► Java uygulamaları geliştirilirken interface oldukça sık kullanılır. Java API bir çok interface içerir.

Interface	Description
Comparable	

Fig. 10.15 | Comrr

Interface	Description
Serializable	
Runnable	

Fig. 10.15 | Comrr

Interface	Description
GUI event-listener interfaces	You work with graphical user interfaces (GUIs) every day. In your web browser, you might type the address of a website to visit, or you might click a button to return to a previous site. The browser responds to your interaction and performs the desired task. Your interaction is known as an <i>event</i> , and the code that the browser uses to respond to an event is known as an <i>event handler</i> . In Chapter 12, JavaFX Graphical User Interfaces: Part 1, you'll begin learning how to build GUIs with event handlers that respond to user interactions. Event handlers are declared in classes that implement an appropriate <i>event-listener interface</i> . Each event-listener interface specifies one or more methods that must be implemented to respond to user interactions.

Fig. 10.15 | Common interfaces of the Java API. (Part 3 of 4.)



► Java uygulamaları geliştirilirken interface oldukça sık kullanılır. Java API bir çok interface içerir.

Interface	Description
Comparable	

Fig. 10.15 | Common interfaces of the Java API. (Part 1 of 4.)

Interface	Description
Serializable	
Runnable	

Fig. 10.15 | Common interfaces of the Java API. (Part 2 of 4.)

Interface	Description
GUI event-listener interfaces	

Fig. 10.15 | Common interfaces of the Java API. (Part 3 of 4.)

Interface	Description
AutoCloseable	Implemented by classes that can be used with the try-with-resources statement (Chapter 11, Exception Handling: A Deeper Look) to help prevent resource leaks. We use this interface in Chapter 15, Files, Input/Output Streams, NIO and XML Serialization, and Chapter 24, Accessing Databases with JDBC.

Fig. 10.15 | Common interfaces of the Java API. (Part 4 of 4.)



- ▶ Java 8 den önce sadece **public abstract** metotlar eklenebiliyordu.
- ▶ Java 8 ile **public default** metot özelliği geldi.
 - ▶ Somut metotlar.
 - ▶ Bir sınıf böyle bir interface implement ettiğinde, default metotları da almış olur.
 - ▶ Bir sınıf böyle bir interface implement ettiğinde, default metotları override etmek zorunda değil, ancak gerekirse bunu yapabilir.
- ▶ Bu özellik ile birlikte interface ve abstract class birbirine yakın hale geldi.
 - ▶ Ancak interface, sınıflarınızı tasarlamada size daha fazla esneklik sağlar, çünkü bir sınıf birçok interface implement edebilir, ancak yalnızca bir üst sınıfı extend edebilir.



- ▶ Java SE 8'den itibaren, yalnızca tek bir soyut metot içeren herhangi bir interface, “functional interface” olarak bilinir;
- ▶ Bu kitapta kullanacağınız işlevsel arayüzler şunları içerir:
 - ▶ ActionListener (Bölüm 12)—Kullanıcı bir düğmeye tıkladığında çağrılan bir metodu tanımlamak için bu interface implement edilir.
 - ▶ Comparator (Bölüm 16)—İlk nesnenin ikinciden küçük, ona eşit veya ikinciden büyük olup olmadığını belirlemek için belirli bir türdeki iki nesneyi karşılaştırabilen bir metot tanımlamak için bu interface implement edilir.
 - ▶ Runnable (Bölüm 23)—Programınızın diğer bölümleriyle paralel olarak çalıştırılabilecek bir görevi tanımlamak için bu interface implement edilir.



- ▶ interface içinde private metot eklenebilir.
- ▶ Bu sayede kod tekrarının önüne geçilecektir.

```
interface Yazdir{  
    default void baslat() {  
        mesaj();  
    }  
    private void mesaj() {  
        System.out.println("Merhaba Dünya");  
    }  
}
```

```
public class Main implements Yazdir{  
    public static void main(String[] args){  
        Main main = new Main();  
        main.baslat();  
    }  
}
```



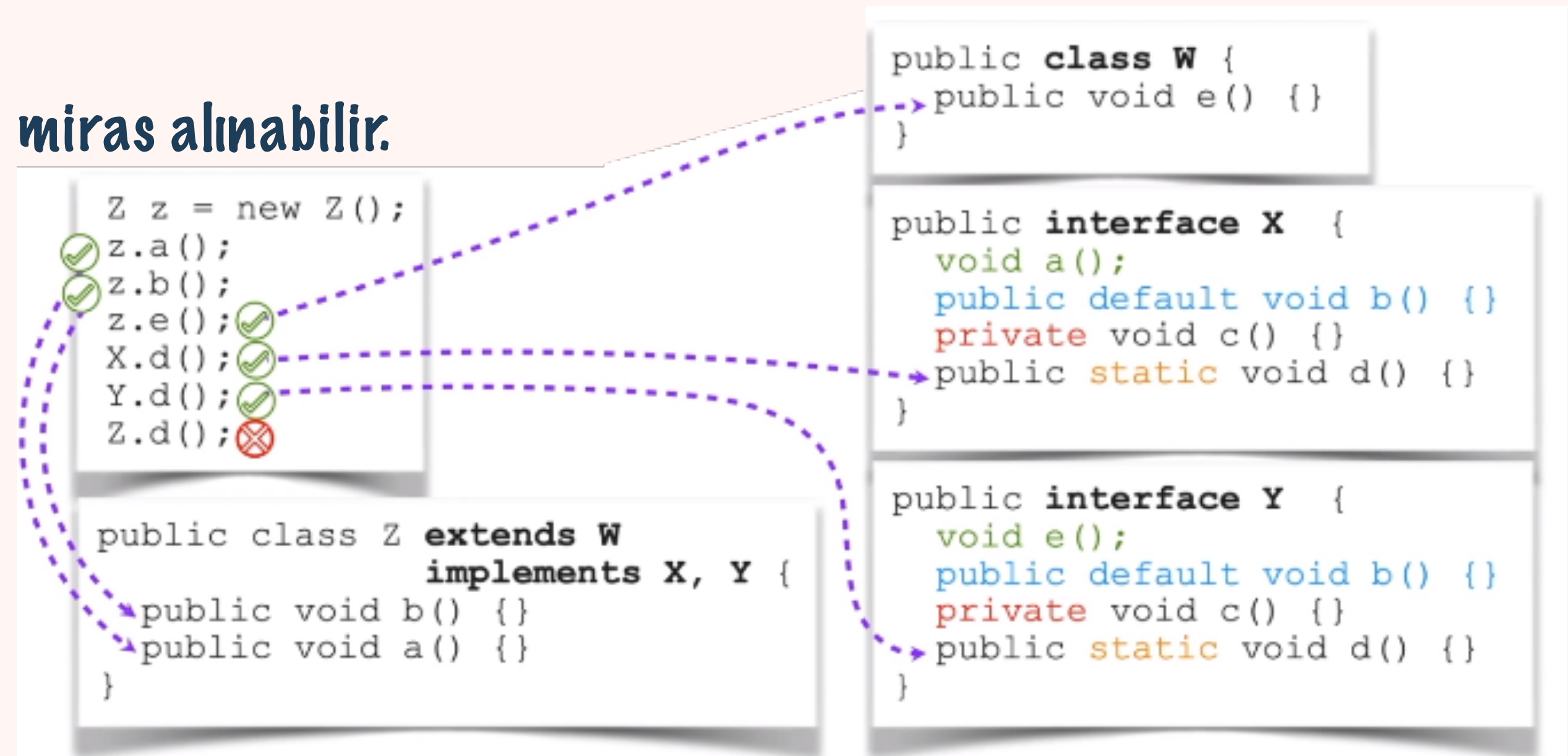

- Bir interface, diğer sınıflarda kullanılabilen bir dizi özellik tanımlar.
- Örnek metotları default olarak **public ve abstract** tır.
- Concrete (somut) metotlar yalnızca **default**, **private** veya **static** ise tanımlanabilir.
- **Constant** içerebilir ancak değişken içeremezler.

```
public interface <InterfaceName> {  
    <constants>  
    <abstract methods>  
    <default methods>  
    <private methods>  
    <static methods>  
}
```

```
public interface Perishable {  
    public static final Period MAX_PERIOD = Period.ofDays(5);  
    void perish();  
    boolean isPerished();  
    public default boolean verifyPeriod(Period p) {  
        return comparePeriod(p) < 0;  
    }  
    private int comparePeriod(Period p) {  
        return p.getDays() - MAX_PERIOD.getDays();  
    }  
    public static int getMaxPeriodDays() {  
        return MAX_PERIOD.getDays();  
    }  
}
```

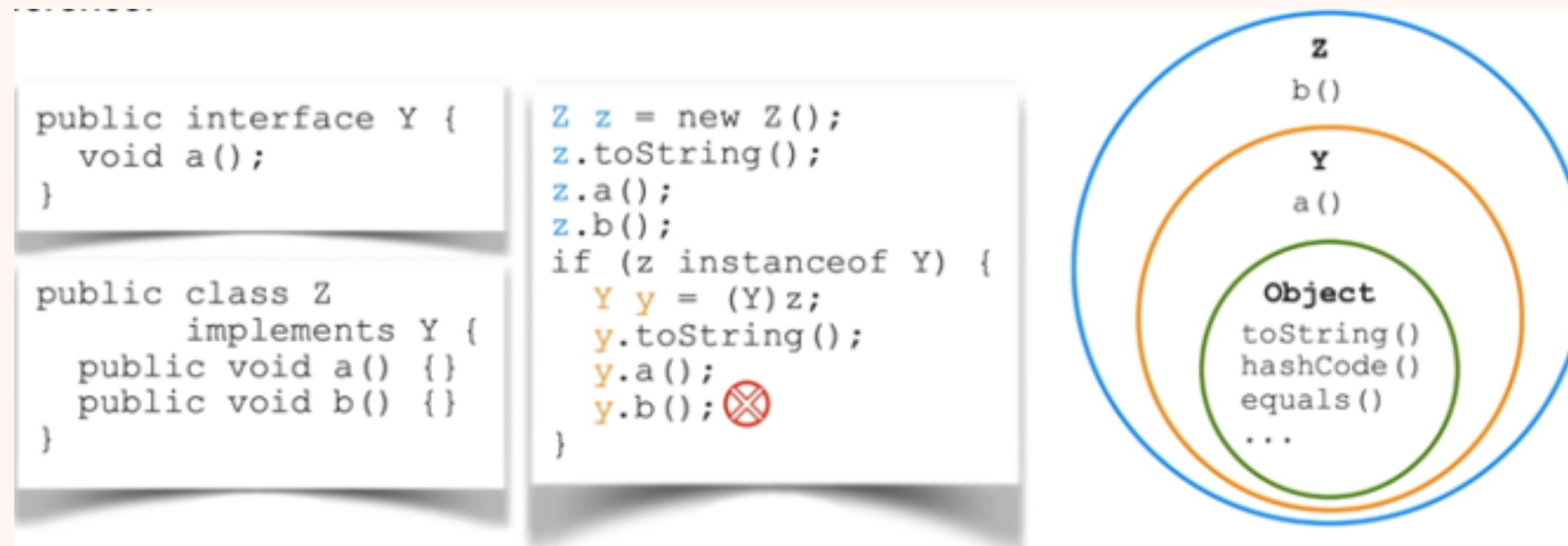



- ▶ Interface içinde somut kodlar yalnızca default, private ve static metotlardar içinde sunulur.
- ▶ Private interface metotlar çakışmaya neden olmazlar, çünkü interface dışında erişilebilir değildir.
- ▶ Static interface metotları, parent tipleri aracılığı ile çağrıldıkları ve süper referansa dayanmadıkları için çakışmalara neden olmazlar.
- ▶ Default metotlar arasında bir çakışma varsa, bu default metodu implemente edilen sınıf içindeki bir metotla override ederek çözümlenmelidir.
- ▶ Aksi takdirde default metot implementasyonu miras alınabilir.

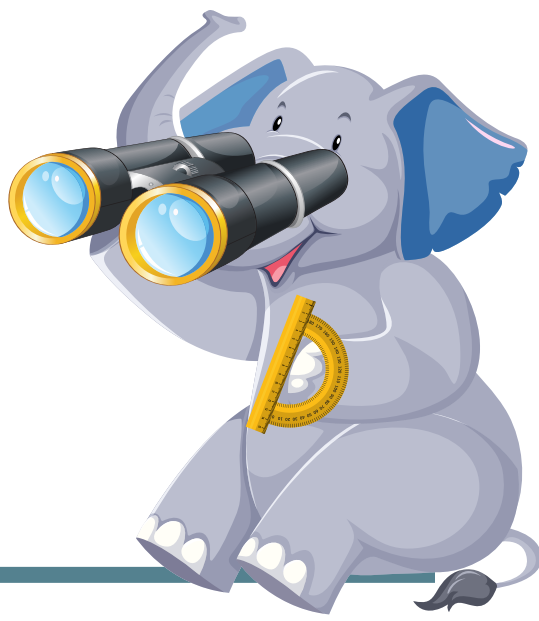




- Bir interface, sınıf gibi bir tiptir.
- Interface geçerli bir referans tipidir, tür dönüşümünde kullanılabilir ve instanceof operatörü ile çalışır.
- Belirli bir sınıf tipi olarak bildirilen referans, parent sınıfları tarafından tanımlanan metotlar dahil olmak üzere bu sınıfın public olarak görülebilen tüm yeteneklerine ve implemente ettiği interface'lere erişim sağlar.
- Interface tipi olarak bildirilen referans, yalnızca bu interface ve parent'leri tarafından tanımlanan işlemlere erişim sağlar.
- Object sınıfı Java'da en son parent sınıftır. Bu nedenle, Object sınıfının metotlarına herhangi bir referans tipi aracılığıyla erişilebilir.



SOFTWARE ENGINEERING OBSERVATION



Interface is a interface.

Abstract class is a class.



	INTERFACE	ABSTRACT CLASS
Metot tipi	Interface yalnızca abstract metotlara sahiptir.	Abstract sınıflar hem abstract hem de abstract olmayan metotlara sahip olabilir.
Final değişkeni	Interface de tanımlanan değişkenler default olarak final'dır.	Abstract sınıflar final olmayan değişkenler alabilir.
Değişken Tipleri	Interface yalnızca static ve final değişkenlere sahip olabilir.	Abstract sınıfta final/ final olmayan, static/static olmayan tüm değişkenler tanımlanabilir.
Implement etme	Abstract sınıf içinde interface implementasyonu yapılabilir.	Ancak interface bir abstract sınıfı implemente edemez.
Kalıtım vs soyutlama	Java interface «implements» anahtar kelimesi ile implement edilir	Abstract sınıflar «extends» anahtar kelimesini kullanır.
Çoklu implement işlemi	Bir interface yalnızca başka bir Java interface'i implemente edebilir	Abstract sınıf ise hem bir Java sınıfını extend eder hem de bir Java interface'i implemente edebilir.